PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT* MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A CONTROLLED COMPARISON OF MANUAL AND GITOPS DEPLOYMENT PARADIGMS IN RESPONDING TO CONFIGURATION DRIFT ON KUBERNETES CLUSTERS



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT* MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A CONTROLLED COMPARISON OF MANUAL AND GITOPS DEPLOYMENT PARADIGMS IN RESPONDING TO CONFIGURATION DRIFT ON KUBERNETES CLUSTERS

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PARADIGMA *DEPLOYMENT* MANUAL DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Dr. techn. Kabul Kurniawan, S.Kom., M.Cs. Ketua Penguji

Aina Musdholifah, S.Kom., M.Kom., Ph.D.
Anggota Penguji

DAFTAR ISI

Halaman Judul **ii**

Halaman Pengesahan **iii**

INTISARI **vi**

ABSTRACT **vii**

I PENDAHULUAN **1**

1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 4
1.6 Sistematika Penulisan 5

II PENELITIAN TERKAIT **6**

2.1 Penyimpangan Konfigurasi (*Configuration Drift*) 6
2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi 7
2.2 Terminologi Dasar 7
2.2.1 Kubernetes dan Orkestrasi Kontainer 7
2.2.2 Git dan *Version Control* 8
2.2.3 *Continuous Integration* dan *Continuous Deployment* 8
2.2.4 *Infrastructure-as-Code* (IaC) 8
2.2.5 Paradigma Deklaratif vs Imperatif 8
2.3 Manajemen Konfigurasi Infrastruktur 9
2.4 *Continuous Deployment* dan Paradigma GitOps 9
2.4.1 Performa *Deployment* dan *Anti-Pattern* 10
2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif . 10
2.5.1 *Convergence* Menuju *Desired State* 11
2.5.2 *Audit Trail* sebagai *Social Control Mechanism* 11
2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan 11
2.7 Paradigma *Deployment* Alternatif 12
2.8 Evaluasi Empiris GitOps dan ArgoCD 12

2.9 Metode Eksperimental Terkontrol pada Riset Sistem 13
2.10 Klasifikasi Beban Uji (*Workload*) 14
2.11 Kesenjangan Penelitian (*Research Gap*) 15

III METODE DAN RANCANGAN PENELITIAN 17
3.1 Deskripsi Umum dan Status Rancangan 17
3.2 Metodologi dan Kesetaraan Perbandingan 17
3.3 Lingkungan dan Provenans Deployment 19
3.4 Intervensi dan Kepemilikan Pengendali 20
3.5 Variabel dan Definisi Operasional 22
3.5.1 R1: Kesetaraan Konfigurasi 22
3.5.2 R2: Deteksi, Bukti Deviasi, dan Pemulihan 22
3.5.3 R3: Bukti Keterlacakan 23
3.6 Gerbang Baseline dan Reset 23
3.7 Instrumen dan Protokol Pengukuran 24
3.8 Pengulangan dan Analisis Statistik 25
3.9 Ancaman terhadap Validitas 25

IV JADWAL PENELITIAN 27
4.1 Penjelasan Kegiatan 27

INTISARI

Perbandingan Terkontrol Paradigma *Deployment* Manual dan GitOps dalam Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy
23/522547/PA/22475

Penelitian ini merancang evaluasi empiris terkontrol atas pengelolaan imperatif terotomasi berbasis `kubectl` dan GitOps berbasis ArgoCD dalam menghadapi penyimpangan konfigurasi Kubernetes. Beban formal adalah InvenioRDM nyata dengan sumber deployment yang dibekukan; surrogat historis tidak menjadi sampel formal. Tujuh skenario direncanakan. Skenario A mempertahankan HPA normal pada kedua lingkungan sebagai kontrol respons HPA, bukan bukti efikasi ArgoCD; skenario F merupakan batas visibilitas objek di luar cakupan aplikasi. Keduanya dilaporkan terpisah dari drift terkelola B–E dan G. Metrik mencakup kesetaraan bidang konfigurasi (R1), waktu deteksi, ketersediaan bukti deviasi dan pemulihan konfigurasi (R2), serta bukti keterlacakan (R3). Kesehatan dan adopsi runtime dinilai terpisah. Target deskriptif adalah lima pengulangan per skenario dan lingkungan untuk R1 serta sepuluh untuk R2, bukan jaminan daya uji. Waktu yang tidak teramati dicatat tersensor, bukan nol; diff bukan identifikasi penyebab kausal. Analisis eksploratif per skenario mempertimbangkan Mann–Whitney U dan ukuran efek untuk observasi lengkap yang sesuai. Rancangan masih menunggu tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan, prasyarat kapasitas, audit, SMTP dan reset aplikasi belum terbukti, dan belum tersedia hasil komparatif formal.

Kata Kunci: penyimpangan konfigurasi, Kubernetes, GitOps, *deployment* manual, evaluasi empiris terkontrol

ABSTRACT

A Controlled Comparison of Manual and GitOps Deployment Paradigms in Responding to Configuration Drift on Kubernetes Clusters

By

Muhammad Argya Vityasy
23/522547/PA/22475

This proposal designs a controlled empirical evaluation of automated imperative management using kubectl and GitOps using ArgoCD under Kubernetes configuration drift. The formal workload is real InvenioRDM with a frozen deployment source; historical surrogates are not formal samples. Seven scenarios are planned. Scenario A retains normal HPA operation in both arms as an HPA-response control, not evidence of ArgoCD efficacy; scenario F examines the visibility boundary for unmanaged objects. Both are reported separately from managed drift scenarios B–E and G. Metrics cover selected-field configuration equality (R1), detection, deviation-evidence availability and configuration recovery times (R2), and traceability evidence (R3). Health and runtime adoption are assessed separately. Descriptive targets are five repetitions per scenario and arm for R1 and ten for R2, not a statistical-power guarantee. Unobserved times are censored rather than set to zero; a diff does not establish a causal root cause. Exploratory per-scenario analysis considers Mann–Whitney U and effect sizes for suitable complete observations. Methodological owner and supervisor review remains pending, formal execution is disabled, capacity, audit, SMTP and application-reset prerequisites remain unproven, and no formal comparative results are available.

Keywords: configuration drift, Kubernetes, GitOps, manual deployment, controlled empirical evaluation

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform *container orchestration* yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta *developer* secara global (Cloud Native Computing Foundation, 2024). Kemampuan Kubernetes dalam mengelola *deployment*, *scaling*, dan *self-healing* menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan *availability* tinggi (Burns et al., 2016). Namun, seiring bertambahnya jumlah layanan yang di-*deploy* pada sebuah klaster, kompleksitas operasional meningkat secara drastis. Aplikasi yang dijalankan pada Kubernetes mencakup berbagai kelas *workload* dengan karakteristik yang berbeda, antara lain: (1) *workload* web tanpa state (*stateless web microservice*), (2) *workload* berbasis data yang menyimpan state (*stateful data service*), serta (3) *workload* komposit multi-service yang terdiri atas banyak komponen yang saling bergantung (Siddiqui et al., 2023; Aghili et al., 2024). Kelas terakhir, seperti platform data repositori skala besar, dapat terdiri dari 16 atau lebih komponen: layanan *web*, *worker* asinkron, *database* relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, enkripsi *secret*, *tunnel* jaringan, kebijakan keamanan, *monitoring system*, dan *backup*.

Pengelolaan *deployment* aplikasi multi-service menggunakan pendekatan manual, yakni melalui perintah `kubectl apply`, *manual scaling*, atau *patching* langsung pada klaster, sering menghasilkan inkonsistensi operasional. Praktik ini rentan terhadap penyimpangan konfigurasi (*configuration drift*), yaitu situasi di mana *state* aktual klaster menyimpang dari *state* yang didefinisikan secara deklaratif (Zhang et al., 2026). Penyimpangan konfigurasi dapat timbul akibat tiga penyebab utama: (1) perubahan langsung pada klaster yang tidak tercatat dalam sistem *version control*; (2) prosedur *deployment* yang tidak konsisten antar *operator*; dan (3) ketiadaan mekanisme deteksi otomatis yang dapat mengidentifikasi deviasi sebelum menimbulkan insiden.

Penelitian empiris oleh Zhang et al. (2026) menunjukkan bahwa 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kuberne-

tes mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root* dan tanpa *resource limits*. Temuan tersebut menunjukkan risiko konfigurasi, tetapi defect statis tidak identik dengan drift: penelitian ini membatasi intervensi pada perubahan terhadap acuan yang telah dibekukan.

GitOps menggunakan repositori Git sebagai *source of truth* dan agen rekonsiliasi untuk menyelaraskan keadaan klaster (CNCF GitOps Working Group, 2021). Studi terdahulu telah membahas perbandingan konfigurasi deklaratif dan imperatif, waktu penanganan drift terhadap Ansible, serta rekonsiliasi GitOps dengan pembanding semiotomatis (Nataraja, 2025; Shrestha and Ali, 2024; Matubber, 2025). Kebutuhan penelitian ini adalah bukti yang sesuai dengan konteks InvenioRDM, kepemilikan pengendali dan definisi pengukuran yang transparan, bukan klaim bahwa seluruh perbandingan terkontrol sebelumnya tidak ada.

Perbandingan tetap bermakna untuk mengukur besarnya perbedaan pada kondisi yang dicatat, bukan mengasumsikan GitOps unggul. Tekanan sumber daya, jadwal rekonsiliasi, hooks aplikasi dan pengendali lain dapat memengaruhi respons. Hasil operasional harus dikaitkan dengan konfigurasi serta batas pengamatan yang benar-benar digunakan (Forsgren et al., 2018).

Kesenjangan tersebut menjadi landasan penelitian yang diuraikan dalam proposal ini: terdapat kebutuhan untuk mengevaluasi secara empiris seberapa besar dampak penyimpangan konfigurasi terhadap karakteristik operasional *deployment* (konsistensi, waktu deteksi/pemulihan, dan *traceability*) ketika berbagai skenario penyimpangan terjadi pada dua paradigma manajemen *deployment* yang berbeda, yaitu manual dan GitOps, melalui evaluasi empiris terkontrol pada *workload* multi-service di klaster Kubernetes. Hasil evaluasi tersebut mengkarakterisasi besarnya dan kondisi perbedaan antara kedua paradigma, yang selanjutnya menjadi dasar rekomendasi pemilihan paradigma *deployment* bagi praktisi industri.

1.2 Rumusan Masalah

Permasalahan yang dievaluasi adalah ketidaksetaraan konfigurasi terhadap acuan, keterlambatan sinyal deteksi dan bukti deviasi, serta keterbatasan jejak perubahan operasional (Zhang et al., 2026; Rahman et al., 2023; Pohjola, 2025). Status rollout sehat tidak menjamin konfigurasi cocok, dan riwayat Git tidak otomatis merekam pelaku mutasi di luar Git.

Pembanding manual pada rancangan ini berarti prosedur imperatif terotomasi

berbasis `kubectl`, bukan eksperimen waktu reaksi manusia. Kedua prosedur dievaluasi pada InvenioRDM nyata dengan aturan kesetaraan yang sama. Kepemilikan HPA normal tetap aktif pada kedua lengan; respons replika HPA tidak dapat dijadikan bukti koreksi ArgoCD atas bidang yang diabaikannya.

Evaluasi mencakup tiga pertanyaan: (1) bagaimana kesetaraan bidang konfigurasi berbeda setelah intervensi; (2) bagaimana waktu deteksi, ketersediaan bukti deviasi dan pemulihan konfigurasi berbeda; serta (3) bukti keterlacakan apa yang tersedia pada kedua lingkungan. Tujuh skenario meliputi perubahan replika (A), image (B), ConfigMap (C), penghapusan objek (D), patch requests/limits (E), namespace salah (F), dan Secret (G). A dilaporkan sebagai kontrol HPA dan F sebagai batas visibilitas objek tak terkelola, terpisah dari efikasi ArgoCD pada B–E/G. Tidak adanya OutOfSync bukan deteksi nol; diff bukan identifikasi akar masalah.

1.3 Batasan Masalah

Agar penelitian tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. Penelitian dilakukan pada klaster Kubernetes tunggal yang dikelola Rancher dengan spesifikasi 3 *node*, 24 total inti CPU, dan $\sim$23 Gi total memori; skenario *multi-cluster* tidak termasuk dalam ruang lingkup.

2. Beban formal adalah InvenioRDM nyata beserta dependensi multi-service; sasaran sedikitnya 16 komponen memerlukan inventaris terverifikasi. Surrogat NGINX historis bukan penggantinya. Inferensi dibatasi pada workload dan apparatus yang benar-benar dicatat, bukan seluruh aplikasi multi-service.

3. Skenario penyimpangan konfigurasi yang diuji terbatas pada tujuh kategori: (A) perubahan replika, (B) perubahan *image*, (C) modifikasi ConfigMap, (D) *resource deletion*, (E) *patch* manual tidak sah, (F) penyimpangan lintas *namespace*, dan (G) perubahan Secret.

4. Dua prosedur dibandingkan: (a) imperatif terotomasi menggunakan `kubectl`, dan (b) GitOps menggunakan ArgoCD. HPA normal aktif pada keduanya. Perubahan metodologis tetap draf untuk tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan sampai seluruh gerbang kesiapan disetujui.

5. Analisis statistik eksploratif menggunakan *Mann–Whitney U test* untuk perbedaan antara kedua lingkungan pada observasi lengkap yang sesuai, dengan tingkat signifikansi $\alpha = 0{,}05$ dan ukuran efek (Cohen's d atau Cliff's δ). Target pengulangan bersifat deskriptif; data tersensor dilaporkan terpisah dan tidak diimputasi sebagai nol.

6. Paradigma perantara (Git sebagai *source of truth* tanpa agen *reconciliation* otomatis) dan paradigma *push-based* (seperti Jenkins CI/CD atau Ansible yang mendorong perubahan ke klaster dari luar) tidak termasuk dalam ruang lingkup. Paradigma perantara tidak memiliki mekanisme koreksi otomatis sehingga tidak memenuhi prinsip GitOps *continuous reconciliation*, sementara paradigma *push-based* memerlukan kredensial klaster yang terekspos dan tidak memiliki *feedback loop* bawaan (Utami and Fatta, 2021; Shrestha and Ali, 2024). Keduanya merepresentasikan titik yang berbeda pada spektrum deklaratif-imperatif dan dapat dievaluasi pada penelitian lanjutan.

1.4 Tujuan Penelitian

Tujuan penelitian adalah mengevaluasi perbedaan prosedur imperatif terotomasi dan GitOps pada kesetaraan konfigurasi, waktu deteksi/bukti deviasi/pemulihan, serta keterlacakan dalam konteks InvenioRDM. Hasil tidak diasumsikan mendukung salah satu prosedur; kontrol HPA dan batas visibilitas tidak digabungkan menjadi klaim efikasi ArgoCD.

1.5 Manfaat Penelitian

Penelitian yang diuraikan dalam proposal ini diharapkan memberikan manfaat berikut:

1. Teoritis: Memberikan bukti empiris terukur mengenai dampak operasional skenario penyimpangan konfigurasi pada dua paradigma *deployment*, yang selama ini banyak didasarkan pada klaim *vendor* atau studi kasus deskriptif. Hasil tersebut melengkapi literatur akademis dengan data eksperimental yang dapat direproduksi dan diverifikasi.

2. Metodologis: Menawarkan protokol eksperimental yang dapat direplikasi, termasuk skenario penyimpangan terstandarisasi, metrik operasional, dan prose-

dur analisis statistik, sehingga penelitian selanjutnya dapat mereplikasi, memperluas, atau mengkritik temuan pada konteks dan platform yang berbeda.

3. Praktis: Menyediakan bukti empiris terkontrol yang mengkarakterisasi perbedaan operasional antara kedua paradigma *deployment* pada berbagai skenario penyimpangan, serta menyoroti skenario penyimpangan yang paling rentan pada masing-masing paradigma, sehingga membantu praktisi industri dalam memilih paradigma *deployment* yang sesuai.

1.6 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- Bab I: Pendahuluan, berisi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, dan sistematika penulisan.
- Bab II: Penelitian Terkait, membahas studi empiris terkait penyimpangan konfigurasi pada Kubernetes, terminologi dasar, manajemen konfigurasi infrastruktur, paradigma *deployment* alternatif, evaluasi GitOps dan ArgoCD, metode eksperimental pada riset sistem, serta *research gap* yang menjadi landasan penelitian.
- Bab III: Metode dan Rancangan Penelitian, menjelaskan metodologi evaluasi empiris terkontrol, definisi variabel dan intervensi, tujuh skenario penyimpangan, protokol pengukuran, analisis statistik komparatif, ancaman terhadap validitas, dan lingkungan eksperimen.
- Bab IV: Jadwal Penelitian, memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 6 bulan.

BAB II

PENELITIAN TERKAIT

2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah situasi di mana *state* aktual sebuah sistem menyimpang dari *state* yang didefinisikan secara deklaratif (Rahman et al., 2020; Hassan et al., 2024). Dalam konteks Kubernetes, penyimpangan terjadi ketika *operator*, baik secara langsung maupun tidak sengaja, mengubah manifest klaster tanpa memperbarui *source of truth* deklaratif, sehingga *state* aktual dan *desired state* menjadi tidak konsisten.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Temuan tersebut didukung oleh Rahman et al. (2019) yang mengidentifikasi tujuh *security smells* berulang pada skrip IaC melalui studi empiris terhadap 1.726 skrip dan 15.232 skrip dari 293 repositori *open-source*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Rahman et al. (2020) memberikan taksonomi empiris untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Melalui analisis kualitatif terhadap 1.448 *commit* terkait *defect* dan kuantifikasi terhadap 80.425 *commit* dari 291 repositori, penelitian tersebut mengidentifikasi delapan kategori *defect*, di mana kategori data konfigurasi yang keliru merupakan kategori paling sering muncul dan kategori *idempotency* merupakan kategori khas IaC yang paling disepakati oleh 66 praktisi yang disurvei. Taksonomi tersebut menyediakan dasar empiris untuk mengoperasionalkan faktor pendorong penyimpangan pada R1, menggantikan kerangka konseptual tanpa validasi Kubernetes-eksklusif.

2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada klaster: Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik (Rahman et al., 2020).

3. Kurangnya mekanisme *rollback* yang terintegrasi: *Deployment* manual seringkali tidak memiliki mekanisme *rollback* otomatis; *operator* harus mengingat konfigurasi sebelumnya atau mengandalkan *backup* yang mungkin tidak konsisten (Rahman et al., 2023, 2019).

4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar de facto untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

2.2.2 Git dan *Version Control*

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth* yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-*review*, dan dikembalikan (*rollback*) secara andal.

2.2.3 *Continuous Integration* dan *Continuous Deployment*

Continuous Integration (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010; Ebert et al., 2016). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

2.2.4 *Infrastructure-as-Code* (IaC)

Infrastructure-as-Code (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reprodusibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* klaster.

2.2.5 Paradigma Deklaratif vs Imperatif

Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (*how to achieve the state*). Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* dan sistem bertanggung jawab mencapai serta mempertahankan *state* tersebut (*what state to achieve*) (Scott, 2015). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual eksplisit untuk setiap deviasi.

2.3 Manajemen Konfigurasi Infrastruktur

Infrastructure state management adalah disiplin yang mengatur bagaimana *state* sebuah sistem infrastruktur didefinisikan, diterapkan, dan dipertahankan. Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* melalui konfigurasi yang telah dibakukan (*Infrastructure-as-Code*), dan sistem bertanggung jawab untuk mencapai serta mempertahankan *state* tersebut (Morris, 2020). Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (Limoncelli et al., 2018). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual untuk setiap deviasi.

2.4 *Continuous Deployment* dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018; Ebert et al., 2016). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

Karakteristik GitOps yang berbeda dari pendekatan imperatif tradisional meliputi: *audit trail* lengkap melalui Git *history*, penerapan *code review* pada perubahan infrastruktur, *rollback* yang sederhana melalui `git revert`, dan reprodusibilitas konfigurasi di berbagai lingkungan (Yuen et al., 2021). Dalam model *pull-based* GitOps, agen yang berjalan di dalam klaster secara periodik membandingkan *state* Git dengan *state* klaster dan menyelaraskan keduanya, menghilangkan kebutuhan akan

akses kredensial klaster dari luar (Utami and Fatta, 2021).

2.4.1 Performa *Deployment* dan *Anti-Pattern*

Forsgren dkk. (Forsgren et al., 2018) mengidentifikasi empat metrik kunci yang mengkarakterisasi performa *deployment* pada organisasi teknologi: (1) *deployment frequency*, (2) *lead time for changes*, (3) *time to restore service*, dan (4) *change failure rate*. Operasionalisasi keempat metrik tersebut untuk pengukuran otomatis dibahas oleh Sallin et al. (2021) melalui prototipe pengukuran otomatis pada XP 2021, sementara Wilkes et al. (2023) memformalkan kerangka pengukuran DORA yang independen terhadap siklus hidup pengembangan dan memvalidasinya pada 304 repositori *open-source*, sehingga mendukung justifikasi pengumpulan metrik berbasis *polling* dalam penelitian ini. Metrik-metrik tersebut menunjukkan bahwa organisasi yang mengadopsi praktik *continuous deployment* dan *Infrastructure-as-Code* secara konsisten mengungguli organisasi yang bergantung pada proses manual. Lebih lanjut, praktik *deployment* manual yang mengabaikan *version control* dan *automated reconciliation* merupakan *anti-pattern* yang berkontribusi terhadap penyimpangan konfigurasi (Beyer et al., 2016). Dalam konteks penelitian yang diuraikan dalam proposal ini, metrik *time to restore service* dan *change failure rate* berelasi langsung dengan metrik R2 (waktu deteksi/pemulihan) dan R1 (konsistensi *deployment*) yang digunakan dalam eksperimen.

ArgoCD mengimplementasikan prinsip-prinsip GitOps melalui *reconciliation loop* yang berjalan di dalam klaster. ArgoCD terdiri dari tiga komponen utama: (1) *application controller* yang secara periodik membandingkan *state* Git dengan *state* aktual dan melakukan tindakan korektif; (2) *repo server* yang mengelola koneksi ke repositori Git; dan (3) *Redis cache* yang menyimpan *state* sementara untuk performa (Argo Project, 2024). ArgoCD berperan sebagai *negative feedback controller*: setiap deviasi (penyimpangan) bersifat *transien*, yaitu sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.5.1 *Convergence* Menuju *Desired State*

Konsep *negative feedback controller* menjelaskan upaya sistem mencapai keadaan yang diinginkan (Limoncelli et al., 2018). ArgoCD membandingkan deklarasi dengan keadaan klaster dan dapat mengoreksi perbedaan sesuai kebijakan, izin dan cakupannya (Argo Project, 2024; Hassan et al., 2024). Hal ini tidak menjamin setiap perubahan ditemukan atau pulih: bidang yang diabaikan, objek tak terkelola, kegagalan pengendali dan kepemilikan HPA membatasi interpretasi. Pada rancangan ini, respons replika HPA dipisahkan dari self-heal ArgoCD.

Perintah imperatif tidak dengan sendirinya menyediakan rekonsiliasi berkelanjutan, tetapi pemeriksaan dan koreksinya dapat diotomasi (Hassan et al., 2024; Rahman et al., 2020). Pembanding penelitian ini menggunakan prosedur otomatis tetap; waktu reaksi manusia tidak dimasukkan.

2.5.2 *Audit Trail* sebagai *Social Control Mechanism*

Git merekam perubahan deklarasi yang dikomit beserta metadata dan dapat mendukung peninjauan perubahan (CNCF GitOps Working Group, 2021). Mutasi langsung pada klaster tidak otomatis masuk riwayat Git. Bukti audit API dan tindakan pengendali tetap diperlukan untuk merekonstruksi pelaku, urutan dan hasil mutasi; kelengkapannya dinilai, bukan diasumsikan.

2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan

Runbook adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem. Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Pada rancangan ini, runbook merupakan prosedur koreksi otomatis berdasarkan perbedaan yang teramati, tanpa menerima label skenario dari injector. Entry point `recovery-runbook.sh` saat ini memvalidasi bukti hasil koreksi dan reset secara offline. Eksekusi langsung tetap dinonaktifkan sampai integrasi lifecycle dan postcondition disetujui. Selesainya diagnosis atau rollout sehat bukan bukti seluruh

konfigurasi maupun data aplikasi telah pulih.

2.7 Paradigma *Deployment* Alternatif

Selain paradigma manual dan GitOps yang menjadi fokus penelitian yang diuraikan dalam proposal ini, terdapat paradigma *deployment* lain yang menempati posisi berbeda pada spektrum deklaratif-imperatif. Pertama, paradigma *push-based Continuous Deployment* (misalnya Jenkins CI/CD) mendorong perubahan dari luar klaster ke dalam klaster melalui *pipeline* otomatis. Meskipun menawarkan otomasi *deployment*, paradigma ini memerlukan kredensial klaster yang terekspos pada *pipeline* eksternal dan tidak memiliki mekanisme *reconciliation* bawaan: ketika *state* klaster menyimpang, *pipeline* tidak secara aktif mendeteksi atau mengoreksi deviasi (Utami and Fatta, 2021). Kedua, manajemen konfigurasi imperatif menggunakan alat seperti Ansible atau Terraform menerapkan perubahan melalui perintah imperatif yang dijalankan dari luar klaster. Shrestha and Ali (2024) mengevaluasi Ansible sebagai alternatif GitOps dan menemukan bahwa meskipun Ansible menyediakan otomasi *deployment*, ia tidak memiliki *continuous reconciliation* dan *audit trail* bawaan yang dimiliki GitOps. Ketiga, paradigma perantara — Git sebagai *source of truth* tanpa agen *reconciliation* otomatis — merepresentasikan titik di mana deklarasi *state* disimpan dalam Git, tetapi koreksi penyimpangan tetap bergantung pada intervensi manual.

Perbandingan difokuskan pada otomasi imperatif tetap dan ArgoCD dengan konfigurasi yang dicatat. Tidak ada klaim bahwa selisihnya menjadi batas atas seluruh alternatif. HPA normal pada kedua lengan merupakan pengendali bersama, bukan perbedaan paradigma.

2.8 Evaluasi Empiris GitOps dan ArgoCD

Utami and Fatta (2021) melakukan analisis perbandingan model *deployment* deklaratif *pull-based* (GitOps dengan ArgoCD) versus *push-based* pada Kubernetes, dan menemukan bahwa model *pull-based* dilaporkan lebih andal karena agen berjalan di dalam klaster dan tidak memerlukan kredensial eksternal. Shrestha and Ali (2024) mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, termasuk waktu induksi, penemuan, dan penanganan drift; pembanding Ansible dan konteks protokolnya berbeda dari prosedur imperatif tetap serta kepemilikan pengen-

dali yang dirancang di sini. Nataraja (2025) melakukan analisis berpusat keamanan terhadap pendekatan deklaratif (GitOps/ArgoCD) dan imperatif (Jenkins/`kubectl`) pada Minikube satu node dengan konfigurasi keamanan yang berbeda, dan melaporkan koreksi penyimpangan yang lebih cepat, *compliance rate* yang lebih tinggi, serta paparan kerentanan yang lebih rendah. Skala satu node dan perbedaan konfigurasi tersebut membatasi transfer temuan ke klaster multi-node dan apparatus penelitian ini.

Di sisi fondasi empiris IaC, Hassan et al. (2024) mengkategorikan 5.110 *defect* rekonsiliasi *state* ke dalam delapan kategori sebagai landasan teori *reconciliation loop*, sementara Rahman and Williams (2019) mengidentifikasi 10 properti kode sumber yang berkorelasi dengan skrip IaC yang cacat dan mendukung pemilihan uji *Mann–Whitney* beserta ukuran efek Cliff's Delta. Matubber (2025) mengukur *reconciliation time*, *scalability*, dan *reliability* GitOps untuk infrastruktur K8s, termasuk *self-healing* terhadap penyimpangan konfigurasi; pembandingnya berupa alur kerja lama yang semi-terotomasi, bukan prosedur imperatif tetap dengan acuan dan kepemilikan pengendali yang sama seperti rancangan ini.

Tabel 2.1 merangkum kelemahan metodologis kunci dari penelitian-penelitian tersebut.

Tabel 2.1: Perbandingan Kelemahan Metodologis Penelitian Terdahulu

Penelitian	Fokus	Kelemahan Metodologis
Shrestha and Ali (2024)	GitOps vs Ansible	Penilaian kualitatif untuk tingkat keparahan penyimpangan; rincian uji formal tidak dinilai dalam ringkasan ini
Nataraja (2025)	Keamanan deklaratif vs imperatif	Minikube satu node dan konfigurasi keamanan berbeda membatasi transfer konteks
Matubber (2025)	*Reconciliation time* GitOps	Pembanding legacy semiotomatis bukan prosedur imperatif tetap penelitian ini

2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Limonce-

lli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Rancangan membandingkan dua prosedur dengan intervensi dan acuan setara serta kendali pengganggu yang didokumentasikan (Wohlin et al., 2012). Pengendalian tersebut tidak menjamin paradigma abstrak menjadi satu-satunya perbedaan; jadwal, implementasi pengendali dan kepemilikan bidang tetap membatasi inferensi.

Analisis eksploratif dapat menggunakan Mann–Whitney U dan ukuran efek untuk waktu teramati lengkap dengan asumsi yang sesuai (Wohlin et al., 2012; Rahman and Williams, 2019). Data tersensor tidak boleh diubah menjadi waktu sukses atau diam-diam dibuang; aturan analisis dan target pengulangan dijelaskan pada Bab III. A dan F tetap terpisah dari efikasi ArgoCD.

2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada klaster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.

2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, TLS, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Beban formal penelitian adalah InvenioRDM nyata, meliputi web, worker/scheduler, PostgreSQL, OpenSearch, Redis, penyimpanan persisten dan infrastruktur pendukung. Sasaran cakupan sedikitnya 16 komponen memerlukan inventaris

dari sumber deployment dan keadaan aktual; daftar konseptual bukan bukti jumlah komponen atau penggunaan MinIO/S3 oleh aplikasi. Beban uji merupakan apparatus studi empiris, bukan produk yang dikembangkan. Temuan dibatasi pada konfigurasi InvenioRDM yang benar-benar digunakan; NGINX S1/S2 historis tidak menjadi pengganti atau dasar generalisasi formal.

2.11 Kesenjangan Penelitian (*Research Gap*)

Berdasarkan tinjauan pustaka yang telah diuraikan, terdapat empat kesenjangan yang saling berkaitan dan menjadi landasan penelitian:

1. Kesenjangan Validitas Konstruk: Studi terdahulu memakai definisi penyimpangan, metrik, dan cakupan yang berbeda. Shrestha and Ali (2024) memakai penilaian kualitatif untuk tingkat keparahan, sedangkan Nataraja (2025) berfokus pada keamanan dan *compliance rate*. Perbedaan operasionalisasi tersebut membatasi perbandingan langsung dengan kesetaraan bidang, kesehatan, dan bukti deviasi yang didefinisikan dalam rancangan ini.

2. Kesenjangan Validitas Internal: Konfigurasi apparatus, skala klaster, pembanding, dan kepemilikan pengendali berbeda antarstudi. Nataraja (2025) memakai Minikube satu node dengan konfigurasi keamanan berbeda, sedangkan Matubber (2025) memakai pembanding legacy semiotomatis. Perbedaan tersebut membatasi transfer atribusi kausal ke konteks InvenioRDM dengan HPA normal pada kedua lengan.

3. Kesenjangan Metodologis: Belum ditemukan evaluasi yang menggunakan kontrak bidang konfigurasi yang sama pada kedua lengan, memisahkan sinyal pengendali dari rollout, serta mempertahankan hasil tersensor dan kegagalan reset pada beban InvenioRDM ini. Rancangan ini ditujukan untuk mengisi kebutuhan tersebut; klaim mengenai hasilnya menunggu data formal.

4. Kesenjangan Kontekstual: Evaluasi GitOps yang ada dilakukan pada konteks dan protokol yang bervariasi. Belum ditemukan evaluasi terkontrol pada apparatus InvenioRDM nyata dengan dependensi multi-service, kepemilikan HPA, serta pemisahan skenario kontrol, drift terkelola, dan batas visibilitas seperti yang dirancang di sini.

Rancangan ini ditujukan untuk mengoperasionalisasi R1/R2 dan rubrik deskriptif R3 dalam konteks tersebut, dengan pengendalian sumber serta pengukuran yang dapat diperiksa. Kontribusi empiris dan besarnya perbedaan baru dapat dinilai setelah metode ditinjau, apparatus divalidasi, dan data formal dikumpulkan.

BAB III

METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum dan Status Rancangan

Penelitian ini merancang evaluasi empiris terkontrol atas pengelolaan imperatif terotomasi berbasis `kubectl` (Lingkungan A, disebut manual dalam judul) dan GitOps berbasis ArgoCD (Lingkungan B). Beban formal adalah InvenioRDM nyata beserta dependensi multi-service, bukan aplikasi NGINX pengganti. Yang dibandingkan adalah dua prosedur operasional pada konfigurasi tertentu, bukan keunggulan universal suatu paradigma.

Revisi metodologis ini merupakan draf untuk tinjauan pemilik penelitian dan pembimbing. Eksperimen formal belum disetujui dan eksekusi langsung instrumen dinonaktifkan. Kelulusan uji offline tidak membuktikan kesiapan klaster, audit, SMTP, kapasitas, ataupun pemulihan data aplikasi. Proposal yang diterima sebelumnya dan artefak historis tetap dapat ditelusuri melalui riwayat versi; perubahan definisi tidak mengubah makna hasil historis menjadi hasil formal.

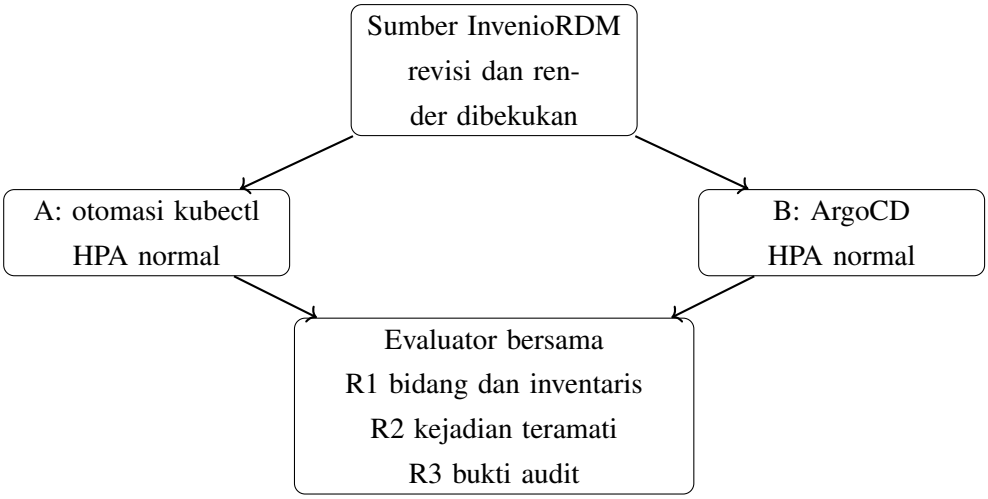
3.2 Metodologi dan Kesetaraan Perbandingan

Rancangan mengikuti evaluasi empiris terkontrol dengan pengamatan kondisi dasar, satu intervensi, dan pengamatan pascaintervensi (Wohlin et al., 2012). Kedua lingkungan dibangun secara sengaja pada satu klaster; rancangan bukan penugasan acak klaster dari populasi (Cook and Campbell, 1979). Unit analisis adalah satu injeksi yang diterima pada satu lingkungan dalam satu pengulangan.

Kedua lingkungan menggunakan sumber deployment, image digest, inventaris, kontrak bidang konfigurasi, kebijakan HPA, dan aturan pengukuran yang setara. Lingkungan manual berada di luar kepemilikan ArgoCD. Urutan lingkungan dan skenario diacak dalam blok; hanya satu injeksi diperbolehkan setelah reset sebelumnya terverifikasi. Tidak ada eksekusi semua mutasi tanpa reset di antaranya.

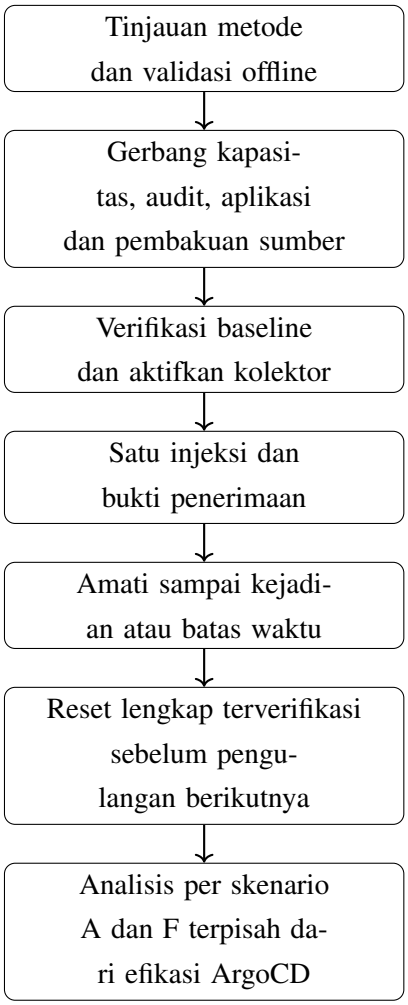
Pemeriksa imperatif membandingkan keadaan aktual dengan acuan yang dibekukan, menyimpan bukti perbedaan, lalu menjalankan koreksi tetap. Tidak ada penundaan untuk reaksi atau pengetikan manusia. Identitas skenario tidak diberikan kepada mekanisme koreksi. Peneliti membakukan prosedur dan meninjau bukti, bukan menjadi subjek pengukuran kinerja manusia; NASA-TLX dan waktu identifikasi

akar masalah manusia tidak menjadi metrik rancangan ini. Prosedur pemulihan tetap mengikuti prinsip runbook yang dapat diperiksa (Schlette et al., 2024).



Gambar 3.1: Rancangan apparatus dengan HPA aktif pada kedua lingkungan.

Gambar 3.1 membedakan mekanisme deployment dari evaluator. HPA merupakan pengendali bersama, sehingga responsnya tidak diatribusikan kepada ArgoCD.



Gambar 3.2: Bagan alir rancangan penelitian; pelaksanaan formal tetap menunggu persetujuan.

Gambar 3.2 menempatkan gerbang kesiapan sebelum injeksi. Kegagalan dan data tersensor disimpan, bukan dihapus agar hanya pemulihan sukses yang dilaporkan.

3.3 Lingkungan dan Provenans Deployment

Konteks deployment yang dilaporkan pemilik adalah tiga node, 24 inti CPU, sekitar 23 Gi memori, Kubernetes 1.32.6 dengan RKE2/containerd, dan Ubuntu 24.04. Nilai ini menggantikan target lama Kubernetes 1.29+ dan Ubuntu 22.04; bukan hasil pemeriksaan live baru dalam revisi ini. Sebelum pengumpulan formal, versi runtime, alokasi node, dan revisi yang benar-benar diterapkan harus direkam ulang.

Sumber otoritatif adalah repositori `https://github.com/vityasyyy/invenio-rdm-gitops`. Revisi referensi yang diperiksa ialah `6cacea6642a7c1690243d3b54130fa08216a9595`; penunjuknya disimpan pada `experiments/deployment-source.json`. Application `argocd/apps/invenio-bootstrap.yaml` memilih `k8s/apps/invenio/kustomization.yaml`, yang merujuk sumber daya dan image digest. Revisi referensi bukan pernyataan bahwa klaster sedang menjalankan revisi itu. Tidak dibuat salinan manifest Invenio editable di repositori penelitian.

Sebelum eksperimen, catat revisi sumber aktual, hasil render beserta SHA-256, image digest, versi ArgoCD dan chart, inventaris objek, penyesuaian namespace/isolasi yang disetujui, serta kepemilikan pengendali. Hasil render merupakan artefak bukti tak berubah, bukan sumber konfigurasi kedua. Dependensi bersama dibedakan dari sumber daya yang diisolasi agar dua lingkungan tidak saling mengubah data.

Konfigurasi sumber `k8s/infra/argocd/patches/argocd-cm.yaml` menetapkan `timeout.reconciliation` sebesar 30 detik, berbeda dari target proposal lama 60 detik. Interval rekonsiliasi repositori ini tidak sama dengan watch kejadian Kubernetes, retry self-heal, atau interval evaluator. Seluruh interval dan perilaku aktual harus dicatat serta dibekukan sebelum formal, bukan disesuaikan setelah melihat hasil.

3.4 Intervensi dan Kepemilikan Pengendali

Pemilihan kategori mempertimbangkan defect konfigurasi Kubernetes dan praktik operasional (Zhang et al., 2026; Pohjola, 2025). Tabel 3.1 membedakan kontrol HPA, drift terkelola, dan batas visibilitas.

Tabel 3.1: Matriks skenario dan batas interpretasi

Skenario	Intervensi	Kepemilikan dan analisis
A	Perubahan replika Deployment	HPA aktif pada kedua lengan; kontrol respons HPA, bukan efikasi ArgoCD
B	Perubahan image	Bidang Deployment terpilih; image digest acuan dibekukan
C	Modifikasi ConfigMap	Kesetaraan objek dan adopsi konfigurasi proses dibedakan
D	Penghapusan sumber daya	Verifikasi penciptaan ulang dan seluruh bidang; bukan sekadar rollout sehat
E	Patch requests/limits	Seluruh bidang terpilih dibandingkan dengan aturan identik
F	Objek pada namespace salah	Batas visibilitas objek tak terkelola; tidak diasumsikan dipangkas ArgoCD
G	Perubahan Secret sintetis	Isolasi dari pengendali SealedSecret wajib ditinjau; nilai tidak diterbitkan

Pada A, HPA web dan worker tetap memiliki replika pada kedua lingkungan. Sumber yang diperiksa mengabaikan `/spec/replicas` pada perbandingan ArgoCD. Karena itu, tidak munculnya *OutOfSync* bukan waktu deteksi nol, dan kembalinya replika bukan bukti self-heal ArgoCD. Nilai acuan dinamis berasal dari keputusan/status HPA yang teramati, bukan angka tetap tiga replika. Kebijakan, metrik, rentang, serta jendela stabilisasi HPA dan penghormatan kepemilikan saat sync harus diverifikasi. Hasil A dilaporkan sebagai kontrol HPA, tidak digabungkan ke estimasi efikasi B–E dan G.

F dilaporkan tersendiri karena objek di luar cakupan Application tidak otomatis menjadi sasaran prune/self-heal. Objek uji harus dibangun dari spesifikasi tersanitasi, bukan menyalin UID, resourceVersion, managedFields atau status objek server ke namespace lain. Penerimaan injeksi dan pembersihan karantina wajib dibuktikan.

Pada G, Secret produksi yang dihasilkan SealedSecret tidak boleh diperlakukan sebagai koreksi ArgoCD murni. Kontrak Secret sintetis dan pengendalinya harus disetujui sebelum formal. Perubahan laten ImageUpdater juga harus dibekukan dalam apparatus yang disetujui. Tidak ada izin mengubah pengendali produksi melalui rancangan ini.

3.5 Variabel dan Definisi Operasional

3.5.1 R1: Kesetaraan Konfigurasi

Kontrak inventaris dan bidang terpilih dibekukan dari sumber yang sama untuk kedua lingkungan. Sumber daya cocok hanya jika ada dan seluruh bidang terpilihnya sama dengan acuan. Kontrak mencakup image, requests/limits, isi ConfigMap, kesetaraan Secret, serta bidang Service dan objek lain yang relevan.

$$R1 = \frac{\text{jumlah sumber daya dengan seluruh bidang terpilih cocok}}{\text{jumlah sumber daya dalam inventaris acuan}} \times 100\%.$$

Status Healthy/Synced bukan pengganti kesetaraan. Kesehatan dan adopsi runtime dilaporkan terpisah. Bidang dinamis HPA dikecualikan dari kesetaraan statis secara eksplisit, bukan dihitung sebagai drift terhadap angka replika tetap. Objek ekstra/karantina dicatat terpisah dari penyebut. Objek hilang pada pembacaan lengkap adalah ketidakcocokan; pembacaan API gagal atau bidang tidak diperiksa adalah kegagalan bukti, bukan skor nol yang sah.

R1 dicatat pada baseline, pengamatan pertama pascainjeksi, dan pascakoreksi. Jika drift sudah pulih sebelum pembacaan, kondisi sesaatnya ditandai tidak teramati. Secret dibandingkan di memori; hanya identitas bidang dan boolean kesetaraan disimpan, tanpa nilai atau hash yang dapat digunakan untuk menebak isinya.

3.5.2 R2: Deteksi, Bukti Deviasi, dan Pemulihan

Semua durasi dihitung dari satu jam monotonik kolektor dalam detik dengan pecahan; UTC disimpan untuk audit. Kolektor harus aktif sebelum injeksi. Interval nominal evaluator satu detik merupakan target validasi, bukan kemampuan live yang telah dibuktikan. Interval aktual dan jeda pembacaan dicatat, terpisah dari jadwal mekanisme koreksi.

- t_0: pengiriman permintaan injeksi; t_a: penerimaan respons API. Injeksi sah hanya jika API berhasil dan perubahan bidang diverifikasi dari respons atau pembacaan balik.
- t_d: sinyal pertama mekanisme pengelolaan yang relevan berdasarkan perbedaan konfigurasi. Pada B–E/G, sumbernya pemeriksa imperatif atau status perbeda-

an ArgoCD sesuai lingkungan. Rollout gagal bukan definisi drift; rollout sehat dapat tetap memiliki image, konfigurasi, atau limits yang salah.

- t_e: waktu pertama bukti tersimpan mengidentifikasi objek dan bidang yang berbeda. Diff menunjukkan deviasi, bukan penyebab kausal; status Healthy dan selesainya perintah diagnosis tidak menghasilkan waktu akar masalah.

- t_r: pengamatan pertama seluruh inventaris dan bidang kembali cocok, dikonfirmasi pada pembacaan berikutnya. Pada A, konfirmasi harus mencakup keadaan dinamis HPA; pada F, ketiadaan objek karantina diperiksa tersendiri.

Durasi utama adalah $t_d - t_0$, $t_e - t_0$, dan $t_r - t_0$. Pada A, sinyal HPA dan respons replika diberi label pengendali HPA, bukan deteksi ArgoCD. Tidak adanya sinyal sampai batas pengamatan dicatat tersensor kanan dengan waktu hilang, tidak diisi nol atau disamakan dengan t_0. Pemulihan konfigurasi tidak membuktikan layanan sehat atau data aplikasi pulih.

3.5.3 R3: Bukti Keterlacakan

R3 dideskripsikan dengan rubrik yang sama untuk pelaku, objek/bidang perubahan, waktu/urutan, dan tindakan/hasil koreksi. Setiap unsur diberi status ada, sebagian, atau tidak ada disertai referensi bukti tersanitasi. Git mencatat perubahan deklarasi, bukan otomatis pelaku mutasi `kubectl` di luar Git. Karena itu, audit API dibutuhkan pada kedua lengan untuk merekonstruksi mutasi tersebut; kelengkapan tidak diasumsikan karena suatu alat digunakan. R3 tidak diuji sebagai perbedaan yang hasilnya telah ditentukan dan tidak menghasilkan klaim akar masalah.

3.6 Gerbang Baseline dan Reset

Sebelum injeksi, inventaris sumber daya, pod dan node wajib tidak kosong serta lengkap; semua pod layanan harus Running dan Ready. CPU dan memori setiap node harus di bawah 70%. Memori didefinisikan sebagai working set metrics-server dibagi allocatable node, bukan metrik host yang dapat dipertukarkan. Metrik tidak tersedia, field tidak terbaca, inventaris parsial, atau status pengendali tidak diketahui menggagalkan gerbang secara tertutup. Kelulusan sampel tunggal tidak menggantikan bukti jendela beban stabil.

Kesiapan juga mencakup bukti audit, SMTP yang benar-benar dapat digunakan, isolasi, sinkronisasi jam, kepemilikan HPA, beban latar, serta pemulihan aplikasi/data/queue yang telah ditinjau. Audit 22 Oktober 2026 menemukan node worker02 melewati 70%; konteks kesiapan yang dilaporkan pemilik tetap sekitar 71%. Kapasitas dua lingkungan Invenio nyata belum terbukti. Audit API belum ditetapkan, SMTP masih placeholder, dan private-draft seed serta recovery aplikasi belum tervalidasi. Tidak ada prasyarat tersebut yang dianggap selesai oleh perubahan instrumen.

Bootstrap, migrasi, fixtures, indexing dan deklarasi queue harus dipisahkan dari repair rutin. Sync perbaikan yang memicu setup dapat menambah kerja dan mengubah baseline. Antarmuka lifecycle wajib mengembalikan exit code, waktu, identitas tindakan dan postcondition; kegagalan tak terduga tidak boleh ditelan. ConfigMap/Secret yang dipakai melalui envFrom memerlukan bukti adopsi runtime terpisah, bukan hanya kesetaraan objek.

Setelah satu pengulangan, reset wajib membuktikan seluruh inventaris/bidang cocok, tidak ada objek karantina, pod siap, kondisi sumber daya memenuhi batas, keadaan HPA sesuai, serta data, queue dan konfigurasi proses kembali ke acuan. Rollout sehat saja tidak cukup. Kegagalan satu tindakan koreksi diteruskan ke pemanggil dan pengulangan berikutnya dilarang sampai reset lengkap terverifikasi. Tidak ada pengosongan queue, modifikasi data, restore atau migrasi langsung yang diizinkan oleh naskah ini.

3.7 Instrumen dan Protokol Pengukuran

Empat entrypoint `baseline-check.sh`, `drift-inject.sh`, `metrics-collect.sh`, dan `recovery-runbook.sh` kini memvalidasi bundel bukti tersimpan secara offline. Pemeriksa bersama menolak bukti hilang/tidak konsisten, menggunakan aturan R1 identik, membedakan sinyal pengendali dari rollout, serta mempertahankan waktu tersensor. Uji fixture mencakup sukses, kegagalan dan batas; fixture bukan hasil pengukuran aplikasi. Adapter polling, injeksi, pemulihan/reset langsung, ekspor render, integrasi lifecycle, audit dan SMTP belum selesai; pengujian formal tidak boleh dimulai sebelum adapter tersebut ditinjau dan divalidasi.

Urutan formal yang direncanakan ialah: (1) bekukan metode, sumber, batas waktu dan aturan analisis; (2) simpan baseline serta aktifkan kolektor; (3) kirim satu injeksi dan verifikasi penerimaan; (4) catat sinyal pengendali, bukti deviasi, keseta-

raan dan kesehatan sampai pulih atau batas pengamatan; (5) simpan kegagalan dan hasil tersensor; (6) verifikasi reset lengkap sebelum pengulangan berikutnya. Tidak ada eksekusi formal dalam revisi ini.

3.8 Pengulangan dan Analisis Statistik

Target deskriptif tetap $n = 5$ per skenario per lingkungan untuk E1/R1 dan $n = 10$ untuk E2/R2; E3/R3 deskriptif. A adalah kontrol HPA dan F batas visibilitas, keduanya dilaporkan terpisah dari B–E/G. Menjumlahkan skenario yang heterogen tidak membuktikan daya uji per skenario. Target ini bukan hasil analisis power yang tervalidasi dan tidak memberi jaminan kecukupan sampel (Cohen, 1988).

Laporkan jumlah injeksi sah, kejadian teramati/tersensor, kegagalan instrumen/pemulihan/reset, median, rentang dan distribusi per skenario. Waktu tidak teramati tetap kosong dengan status sensor dan batas pengamatan. Kegagalan API/injeksi/infrastruktur dibedakan dari tidak terjadinya deteksi; data tidak dibuang tanpa jejak. Rencana uji tetap Mann–Whitney U dua sisi pada $\alpha = 0{,}05$, dengan ukuran efek Cohen's d atau Cliff's δ sebagaimana proposal sebelumnya. Penerapannya memerlukan observasi lengkap serta asumsi yang sesuai; revisi ini tidak menetapkan pilihan ukuran efek atau keluarga uji baru. Jika ada penyensoran, analisis yang mengabaikannya tidak dianggap perbandingan lengkap; keputusan analisis data tersensor dan penyelarasan rincian proposal–skripsi memerlukan tinjauan pemilik/pembimbing sebelum formal.

3.9 Ancaman terhadap Validitas

Validitas internal dipengaruhi perbedaan jadwal pengendali, tekanan node, I/O latar, dependensi bersama, hooks lifecycle, dan kepemilikan HPA/SealedSecret/ImageUpdater. Kendali dan pencatatannya membatasi bias, tidak menjamin satu-satunya perbedaan adalah paradigma abstrak. Validitas konstruk dibatasi kontrak bidang, resolusi pengamatan, bukti audit, serta perbedaan kesetaraan konfigurasi dengan kesehatan dan pemulihan aplikasi. A dan F tidak dapat digunakan untuk klaim efikasi self-heal ArgoCD.

Validitas eksternal dibatasi satu klaster, InvenioRDM, ArgoCD dan konfigurasi yang akhirnya digunakan. Surrogat S1/S2 serta hasil mock historis bukan sampel formal dan tidak memperluas inferensi. Reliabilitas memerlukan provenans, jam ko-

lektor, bukti lengkap, dan reset yang dapat diulang. Sampai tinjauan metode serta seluruh gerbang dipenuhi, belum tersedia dasar untuk kesimpulan komparatif empiris.

BAB IV
JADWAL PENELITIAN

Penelitian direncanakan untuk dilaksanakan selama enam bulan, dari bulan Juni 2026 hingga November 2026. Jadwal penelitian disusun berdasarkan tahapan evaluasi empiris terkontrol: studi literatur, perancangan lingkungan, pengembangan skrip otomasi, kalibrasi *baseline*, pelaksanaan eksperimen, analisis statistik, dan penulisan laporan. Tahapan-tahapan tersebut dipersempit dan saling tumpang tindih untuk efisiensi waktu.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol • menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan					
	Jun	Jul	Agu	Sep	Okt	Nov
Studi Literatur & Perancangan Lingkungan	•	•				
Implementasi ArgoCD + *Baseline* Manual		•	•			
Pengembangan Skrip Otomasi		•	•			
Kalibrasi *Baseline* & Uji Coba			•	•		
Eksperimen & Pengumpulan Data (E1–E3)				•	•	
Analisis Statistik & Interpretasi					•	•
Penulisan Laporan	•	•	•	•	•	•
Seminar / Sidang						•

4.1 Penjelasan Kegiatan

1. Studi Literatur dan Perancangan Lingkungan (Juni – Juli 2026)

   Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai penyimpangan konfigurasi pada Kubernetes, evaluasi empiris GitOps, metode eksperimental pada riset sistem, serta klasifikasi *workload* multi-service. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), dokumentasi resmi, buku teks, dan artikel teknis. Tahap ini juga mencakup perancangan arsitektur Lingkungan A (*deployment* manual/`kubectl`) dan Lingkungan B (*deployment* Gi-

tOps/ArgoCD), definisi variabel dan intervensi, pemilihan tujuh skenario penyimpangan, serta spesifikasi protokol pengukuran.

2. Implementasi ArgoCD dan *Baseline* Manual (Juli – Agustus 2026)

   Tahap implementasi meliputi konfigurasi ArgoCD pada klaster, pembuatan repositori GitOps dengan manifest deklaratif, konfigurasi *baseline* manual (*workflow* `kubectl`), serta konfigurasi *logging* dan *monitoring* (Prometheus, Grafana, Kubernetes *audit logs*, ArgoCD *notifications*). Pada akhir fase ini, kedua lingkungan siap untuk eksperimen.

3. Pengembangan Skrip Otomasi (Juli – Agustus 2026)

   Pengembangan empat skrip otomasi yang mendukung konsistensi prosedural eksperimen: (a) `drift-inject.sh` untuk injeksi skenario penyimpangan secara acak dengan pencatatan *timestamp*; (b) `metrics-collect.sh` untuk pengumpulan metrik secara berkala via ArgoCD API dan `kubectl`; (c) `baseline-check.sh` untuk verifikasi kondisi klaster sebelum setiap *run*; dan (d) `recovery-runbook.sh` untuk memandu *operator* pada Lingkungan A melalui prosedur pemulihan terstandarisasi. Skrip dikembangkan secara paralel dengan implementasi lingkungan.

4. Kalibrasi *Baseline* dan Uji Coba (Agustus – September 2026)

   Sebelum eksperimen dimulai, fase kalibrasi dilakukan untuk memastikan validitas internal. Kegiatan meliputi: (a) *deployment* beban uji (*workload*) komposit multi-service pada kedua lingkungan; (b) verifikasi *baseline* bersih; (c) pengukuran awal konsistensi; (d) verifikasi penggunaan sumber daya *node* di bawah 70%; dan (e) uji coba awal untuk memvalidasi protokol pengukuran dan skrip otomasi, termasuk *practice run operator*.

5. Eksperimen dan Pengumpulan Data (September – Oktober 2026)

   Rencana pelaksanaan tiga eksperimen menunggu tinjauan metode dan kelulusan gerbang Bab III. E1 menilai kesetaraan konfigurasi ($n = 5$ per skenario); E2 waktu deteksi, bukti deviasi dan pemulihan ($n = 10$); E3 bukti keterlacakan secara deskriptif. Target tersebut bukan jaminan daya uji. A dilaporkan sebagai kontrol HPA dan F sebagai batas visibilitas, terpisah dari B–E/G. Urutan diacak, satu injeksi diikuti reset lengkap; kegagalan dan hasil tersensor tetap disimpan. Eksekusi formal belum diizinkan.

6. Analisis Statistik dan Interpretasi (Oktober – November 2026)

   Analisis data meliputi statistik deskriptif (*mean*, median, rentang, deviasi standar), uji perbandingan (*Mann–Whitney U test*), perhitungan ukuran efek (Cohen's *d*), dan interpretasi hasil. Pembahasan mencakup identifikasi skenario penyimpangan yang paling berdampak, faktor yang memengaruhi hasil, serta perumusan rekomendasi pemilihan paradigma *deployment* berbasis bukti empiris terkontrol.

7. Penulisan Laporan (Juni – November 2026)

   Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan periode, draft skripsi lengkap pada akhir penelitian.

8. Seminar / Sidang (November 2026)

   Presentasi dan sidang tugas akhir di depan dosen penguji, mencakup paparan hasil eksperimen, analisis statistik, rekomendasi paradigma, dan kesimpulan.

DAFTAR PUSTAKA

R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.

Argo Project. Argocd documentation, 2024. URL `https://argo-cd.readthedocs.io`.

Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.

Cloud Native Computing Foundation. Cncf cloud native definition v1.0, 2024. URL `https://www.cncf.io/about/who-we-are`.

CNCF GitOps Working Group. Opengitops principles v1.0.0. `https://opengitops.dev`, 2021.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.

Christof Ebert, Gorka Gallardo, Josune Hernantes, and Nicolas Serrano. Devops. *IEEE Software*, 33(3):94–100, 2016. doi: 10.1109/MS.2016.68.

Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Md Mahadi Hassan, John Salvador, Shubhra Kanti Karmaker Santu, and Akond Rahman. State reconciliation defects in infrastructure as code. *Proceedings of the ACM on Software Engineering*, 1(FSE):1865–1888, 2024. doi: 10.1145/3660790.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.

O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.

A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.

Akond Rahman and Laurie Williams. Source code properties of defective infrastructure as code scripts. *Information and Software Technology*, 112:148–163, 2019. doi: 10.1016/j.infsof.2019.04.013.

Akond Rahman, Chris Parnin, and Laurie Williams. The seven sins: Security smells in infrastructure as code scripts. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 164–175. IEEE/ACM, 2019. doi: 10.1109/ICSE.2019.00033.

Akond Rahman, Effat Farhana, Chris Parnin, and Laurie Williams. Gang of eight: A defect taxonomy for infrastructure as code scripts. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering (ICSE)*. ACM/IEEE, 2020. doi: 10.1145/3377811.3380409.

Marc Sallin, Martin Kropp, Craig Anslow, James W. Quilty, and Andreas Meier. Measuring software delivery performance using the four key metrics of devops. In *Agile Processes in Software Engineering and Extreme Programming (XP 2021)*, pages 103–119. Springer, 2021. doi: 10.1007/978-3-030-78098-2_7.

D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.

R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.

H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Brennan Wilkes, Alessandra Maciel Paz Milani, and Margaret-Anne Storey. A framework for automating the measurement of devops research and assessment (dora) metrics. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2023. doi: 10.1109/ICSME58846.2023.00018.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.

Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 52(3):1125–1144, 2026. doi: 10.1109/TSE.2026.3659785.